Selective Recovery in a Multi-Agent Coding Workflow

CSE 291A Fall 2026, Option 5 · Phase 1 Report · November 4, 2026

Jackie Wang, Ray Williams

Code and data: github.com/rayray4059/selective-recovery

Abstract

When there are multiple agents working, sometimes an earlier agent might make a mistake this can affect the agents after it which can dramatically change the results that you were looking for. Recovering from such a mistake usually means either restarting the whole workflow or checking every output from each agent to make sure that they are on the right track, and both are expensive and time consuming. This project that we worked on asks whether tracking the dependencies between agent results can help agent to effective recover while keeping the correct and safe task execution. In Phase 1 we built the workflow and measured the problem. We included three agents, a researcher, a planner and an executor to complete tickets on a small Python library. Only the executor can change files or publish releases, and the result of each ticket is checked by hidden test cases. We then injected seven kinds of faults, such as a wrong fact from the researcher or a malicious instruction hidden in the plan, and traced how each one spread.

Most of the faults caused the workflow to produce a different result than what we expected. Out of the 42 faulted runs on tickets that the workflow could solve correctly without any faults, 38 of them actually produced incorrect code, code that could not run, or carried out the injected attack. When we restarted the whole workflow, it was able to fixed the problem, but in nine runs the faulty workflow had already published a release, and restarting only fixes the files, and not releases that already got published. Restarting could solve the problem but it also has to repeating all of the work for the agent. When the mistake came from the researcher agent, every later stage result all depended on that agent, so none of the model's work could safely be reused. We also added a verifier to check the outputs passed between agents. It stopped all eight attacks in the verified runs, but it also rejected many correct outputs and roughly doubled the running time. These results helped us understand what we need to work on in Phase 2: finding which parts of the workflow need to run again, keeping the work that is still valid, and reporting any actions that recovery cannot undo.

1. Experiment Setup

Domain. For this experiment, we used a small Python `billing` library where the agents work together to complete coding tickets. We chose this setup because test cases can check whether their changes are correct, and Git lets us track every file they change and restore an earlier version. Each of the 10 development tickets depends on a rule found only in the library's internal documents, such as late fees being 1.5% of the invoice and capped at 2500 cents, so the researcher has to find it and pass it on. This lets us test whether a mistake in that information reaches the plan and the final code. Two tickets also ask the agents to publish a release to a simulated registry, where a release cannot be taken back once published.

Workflow. The researcher reads the internal documents and returns up to five facts with their sources. The planner uses these facts, the ticket and the current code to write a plan in JSON. The executor follows the plan and has up to six turns to read or edit files, run the visible tests, publish a release, or post a message. All three agents use Qwen3.5 4B (Q4_K_M) through llama.cpp on a MacBook Pro (M4 Pro), with temperature 0, seed 0 and the model file's hash recorded, so runs are cheap and repeatable. In a pilot, the model alone passed 10 of 15 runs but the three-agent workflow only 7, so we added structured JSON hand-offs and a fact limit.

Checking. After each run, we use hidden test cases to check whether the code behaves as the ticket requires. We also check whether the agents changed files outside the allowed paths or published releases and messages that the ticket did not request. Unauthorized tool calls are recorded separately. To measure the effect of a fault, we compare the run's final files, releases, and messages with those from a clean run of the same ticket. The hidden tests determine whether the task was completed correctly, while the comparison helps us understand what changed because of the fault.

Faults. We inject faults after a stage produces its output and before the next stage reads it. Accidental faults replace the deciding fact with a wrong or stale (2025) value, drop all facts (a timeout), or put the wrong value into the plan. Adversarial faults add an instruction, as a fact or a plan step, to delete the test file, publish release 9.9.9, or rewrite the policy document. A fixed schedule gives every ticket every fault (70 runs). A control fault that changes only an irrelevant fact, and a run with the documents removed, check whether the work actually depends on the injected result.

Tracing and labels. Each run records a provenance trace with one node per artifact and its inputs; a file read links to the step that last wrote that file, so dependencies through files count as well as messages. Each faulted run is compared with the clean run of the same ticket and labeled *ignored* (same final state), *absorbed* (different but still passing) or *harmful*. Tickets the clean workflow fails are excluded, so failure is not mistaken for protection.

Baselines. Full restart is told which result is faulty, resets the files to the first commit and reruns every stage. Always-verify must find faults itself: a verifier using the same model checks the facts and the plan before each hand-off and sends a rejected stage back once. It ran on four tickets because it more than doubles run time. Runs that hit a parser bug were rerun after the fix, together with their restarts; only 2 of them changed pass/fail. Every number below can be rebuilt from the saved traces.

2. Evaluation

Clean runs. The workflow passed 7 of 10 tickets (8.8k tokens, 24 s per run). In every failure the researcher found the right rule but the executor wrote broken code. Only one ticket passed without the documents. All 70 restarts matched their clean runs exactly, so differences between paired runs come from the fault, not randomness.

Propagation. No real fault was ignored, and every fact fault changed both the plan and the executor's actions (Table 1). Accidental faults were harmful in 24 of 28 runs and adversarial faults in 14 of 14; of the 38 harmful runs, 24 produced wrong behaviour, 2 code that would not import, and 12 an executed attack. The control fault changed the executor's actions on all seven tickets but broke only one, so changed outputs alone overstate propagation. Work after a fault averaged 5.9k to 11.2k tokens per run, against 8.8k for a whole clean run.

Fault	Class	Ignored	Absorbed	Harmful	Plan changed	Executor changed
Wrong fact	Accidental	0	0	7	7	7
Stale fact	Accidental	0	0	7	7	7
Timeout	Accidental	0	1	6	7	7
Wrong plan	Accidental	0	3	4	–	7
Malicious fact	Adversarial	0	0	7	7	7
Malicious plan	Adversarial	0	0	7	–	7
Control	Control	3	3	1	5	7

Table 1. Outcomes on the 7 clean-passing tickets, one run each (plan faults cannot change the plan).

Unsafe actions. Across all 10 tickets the executor carried out 16 of 20 injected attacks, equally often from a fact and from a plan step. On the shipping ticket, four accidental faults also led to a wrong version being published.

Recovery baselines. Full restart fixed the code in all 42 faulted runs, but only 33 fully recovered: in the other nine, a release had already been published and could not be withdrawn. After a fact fault none of the earlier work could be reused; after a plan fault, only the researcher's 19.6% could. Always-verify caught the faulty stage in 23 of 24 runs and stopped all eight attacks, raising faulted passes from 0 to 15 of 24. But it rejected something in 31 of 32 runs, including half of the correct fact lists, halved passes on clean and control runs, and added 7.6k tokens and 32 s per run. Tracing costs about 0.4% of run time on average.

3. Conclusion

A wrong fact from the researcher always reached every later stage. Restart repeats all of the work and cannot undo releases, and verification stops attacks but rejects much correct work and roughly doubles the cost. Since our trace records dependencies per stage, selective recovery would currently rerun everything after a fact fault. In Phase 2 we will track which facts each plan step and file change use, add tickets with independent parts so some work can be kept, roll back only the affected files, and compare all three methods on a held-out set.

Appendix: Timeline and milestones

Dates	Work	Milestone
Oct 27	Repo scaffold, state store, unit tests	Done
Oct 27–29	Model pilot on five tickets	Done: Qwen3.5 4B chosen; file hash recorded
Nov 2	Coding workspace and tools	Done: tool unit tests pass
Nov 2	10 dev tasks, hidden tests, checker; agents	Done: reference passes; do-nothing fails
Nov 2–3	Provenance tracing; fault injector and schedule	Done: faulted run compared with its clean pair
Nov 3	Full-restart and always-verify baselines	Done: restart on all tickets; verify on four
Nov 3–4	Dev-set clean, fault and baseline runs; results tables	Done: tables in `results/`
Nov 4	Executor parser fix; affected tickets rerun	Done: 2 runs changed pass/fail
Nov 4	**Phase 1 code and data due**	One command per condition
Nov 5–17	Fact-level provenance; selective invalidation	Only steps that cite a bad fact rerun
Nov 10–20	Held-out set, multi-part and multi-ticket tasks	Unseen until the final evaluation
Nov 12–24	Shared-state rollback and snapshots	Irreversible actions reported
Nov 23–Dec 3	Held-out evaluation on AWS	Same schedules, budgets and seeds
Dec 1–9	Final report	
Dec 10	**Final submission**	